



Universidad Nacional Experimental De Guayana

Vicerrectorado Academico

Coordinacion General De Pregrado

Ingeniería En Informática

Sistema de Bases de Datos II

Proyecto MongoDB

Docente:

Clinia Cordero

Integrantes:

Jose Miserol V24.482.932

Miguel Gomez V29.862.177

Anthony Medina V30.857.435

Ciudad Guayana, Diciembre de 2025

Introduccion

El panorama de la gestión de datos ha evolucionado significativamente más allá del modelo relacional tradicional. En la era del Big Data, las aplicaciones web escalables y la necesidad de flexibilidad en los esquemas, surgieron las bases de datos NoSQL (acrónimo de "Not Only SQL"). Estas representan una clase diversa de sistemas de gestión de bases de datos diseñados para manejar grandes volúmenes de datos con una estructura no relacional, ofreciendo una alternativa potente al modelo SQL.

Las bases de datos NoSQL se distinguen por su capacidad de escalabilidad horizontal, permitiendo distribuir la carga de trabajo entre múltiples servidores, algo fundamental para el crecimiento masivo de datos. Además, su principal ventaja reside en la ausencia de un esquema fijo (schema-less), lo que proporciona a los desarrolladores una gran agilidad para iterar y adaptar el modelo de datos sin complejas migraciones.

A diferencia del estricto cumplimiento del principio ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) del modelo relacional, muchas bases de datos NoSQL priorizan la disponibilidad y la tolerancia a particiones, adhiriéndose a menudo al teorema CAP (Consistencia, Disponibilidad, Tolerancia a Particiones). Esto se traduce en una consistencia eventual en muchos casos, donde los datos se sincronizan con el tiempo.

GitHub. <https://github.com/jose-miserol/globalmarket-analytics-search-engine>

Fase I - Diseño e Ingesta de Datos

Selección y Análisis del Dataset

El equipo seleccionó el dataset "Amazon Sales Dataset" disponible en Kaggle, que contiene información detallada sobre productos, usuarios, reseñas y transacciones de una plataforma de comercio electrónico. Este dataset resultó ideal para el proyecto debido a su volumen significativo de registros (más de 1,000 documentos) y su estructura que permite explorar diferentes patrones de modelado NoSQL. La elección se fundamentó en la presencia de relaciones complejas entre entidades y la variabilidad de atributos entre productos, características que coinciden directamente con los desafíos planteados en el escenario "GlobalMarket".

El análisis preliminar del dataset reveló la necesidad de transformar datos originalmente estructurados para un modelo relacional hacia un esquema orientado a documentos. Se identificaron cuatro entidades principales: productos, usuarios, reseñas y ventas. Cada entidad presentaba particularidades que requerían decisiones estratégicas de modelado, especialmente en lo referente a la gestión de atributos variables y relaciones entre colecciones.

Fundamentos y Mejores Prácticas de Modelado NoSQL

Para la migración de la plataforma "GlobalMarket" desde una arquitectura relacional hacia MongoDB Atlas, el equipo adoptó una metodología de Diseño Basado en Consultas (Query-Driven Design). A diferencia del modelo relacional normalizado que prioriza evitar la redundancia de datos, el enfoque en MongoDB prioriza el rendimiento de las operaciones más frecuentes y críticas del negocio: el motor de búsqueda de productos y el dashboard de análisis de ventas. Siguiendo las mejores prácticas documentadas por MongoDB, el modelo de datos se construyó considerando tres pilares fundamentales: la estructura de los datos, los patrones de acceso de la aplicación y la naturaleza de las relaciones entre entidades.

El primer paso en el modelado consistió en identificar las operaciones dominantes mediante un análisis exhaustivo de la carga de trabajo. Dado que GlobalMarket es una plataforma de comercio electrónico, se determinó que la carga de trabajo es mayoritariamente de lectura intensiva. Los usuarios visualizan catálogos y detalles de productos con mucha más frecuencia de la que realizan compras o escriben reseñas. Esta característica fundamental guió todas las decisiones posteriores de diseño, priorizando estructuras que optimicen las consultas del Aggregation Framework requeridas, como el reporte de ventas por categoría y la identificación de productos destacados.

La decisión crítica entre embeber documentos o usar referencias se tomó evaluando cuidadosamente la cardinalidad y el comportamiento de los datos. Para optimizar la lectura, el

equipo optó por embeber datos relacionados dentro del documento principal siempre que fuera posible, reduciendo así la necesidad de operaciones costosas similares a los JOINS relacionales. En el catálogo de productos, los atributos variables y las especificaciones técnicas se embebieron dentro del documento del producto, solucionando el problema de variabilidad de atributos que afectaba a la base de datos relacional anterior. Sin embargo, se mantuvieron referencias normalizadas únicamente cuando la frecuencia de actualización de los datos relacionados es alta o cuando el tamaño del documento embebido amenazaba con exceder el límite de 16MB de BSON.

Considerando que MongoDB garantiza atomicidad a nivel de documento, el diseño del esquema para el registro de ventas agrupa la información de la transacción completa (cliente, productos, total) en un solo documento. Esto asegura la integridad de la orden de compra sin necesidad de transacciones complejas entre múltiples tablas, facilitando la ingesta de datos y el análisis posterior en tiempo real.

Estrategia de Modelado: El Poder del Embedding

Siguiendo la documentación oficial de MongoDB sobre modelos de datos embebidos, el equipo implementó un esquema desnormalizado donde los datos relacionados se almacenan en una sola estructura de documento. Esta decisión se fundamenta en la capacidad de recuperar datos relacionados en una única operación de base de datos, eliminando la latencia asociada a múltiples consultas o joins complejos.

Para "GlobalMarket", el patrón de embedding se aplicó en dos áreas críticas identificadas durante el análisis de requisitos. El primer caso aborda la variabilidad de atributos en el catálogo de productos, donde el problema principal de la base de datos relacional anterior era la dificultad para manejar productos con atributos dispares. Una laptop tiene CPU y RAM, mientras que una zapatilla tiene talla y color. La solución implementada utiliza documentos embebidos para agrupar estas características bajo campos flexibles como specs o attributes, permitiendo que cada documento de producto tenga una estructura adaptable sin necesidad de alterar el esquema global ni crear tablas de metadatos poco eficientes.

El segundo caso crítico involucra el manejo de reseñas, donde se evaluó cuidadosamente la relación entre producto y reseña. La decisión de embeber las reseñas como un array de sub-documentos dentro del documento del producto se justifica por múltiples factores. Según las mejores prácticas, el embedding es ideal cuando los sub-documentos se visualizan siempre en el contexto del documento padre. Al cargar la página de un producto, la aplicación necesita mostrar sus reseñas inmediatamente, y tener esta información dentro del mismo documento permite

obtener toda la información necesaria para la vista de detalle con una sola lectura en disco, maximizando el rendimiento del dashboard de análisis en tiempo real.

El equipo es consciente del límite de 16MB por documento BSON. Sin embargo, dado que el requerimiento inicial es para un MVP y análisis, y el promedio de reseñas por producto no desbordará este límite en el corto plazo, el beneficio de lectura supera el riesgo de tamaño. Para productos con popularidad extrema en el futuro, se podría refactorizar a un modelo híbrido que combine embedding para reseñas recientes y referencias para el historial completo.

Integridad y Manejo de Redundancia

En el diseño de "GlobalMarket", el equipo adoptó una estrategia de duplicación de datos intencional para optimizar la lectura, alineada con la documentación de MongoDB sobre manejo de datos duplicados. El caso de uso principal involucra la creación de snapshots de productos en órdenes de compra. Al crear una orden, se duplican ciertos datos del producto como nombre, precio y SKU dentro del documento de la orden. Esta duplicación no es un error sino una característica necesaria del Snapshot Pattern para mantener la integridad financiera. Si el precio de un producto cambia posteriormente, el registro histórico de una venta pasada no debe cambiar, asegurando consistencia en los reportes financieros sin depender de consultas complejas de versionado.

Para manejar la consistencia entre datos duplicados que sí cambian, el equipo definió dos niveles de consistencia. Se aprovecha la garantía de MongoDB de que las escrituras en un solo

documento son atómicas para mantener datos críticos como el inventario únicamente en la colección de productos, evitando condiciones de carrera y sobreventa. Para datos duplicados menos críticos, como nombres de categorías embebidos en productos para facilitar búsquedas, se acepta una consistencia eventual donde tareas programadas o triggers de Atlas pueden actualizar los productos relacionados en segundo plano sin bloquear las operaciones de venta principales. La decisión de duplicar datos específicos del vendedor dentro del documento del producto reduce la necesidad de usar operaciones de lookup en la página principal de búsqueda. Esto cumple con la práctica recomendada de optimizar para el caso de uso más frecuente: los usuarios buscan productos por nombre de tienda mucho más seguido de lo que las tiendas cambian de nombre. Por tanto, el costo de escritura ocasional se justifica por la ganancia masiva en velocidad de lectura diaria.

Garantía de Calidad de Datos: Schema Validation

Aunque la flexibilidad de MongoDB es una ventaja clave para el catálogo de productos de "GlobalMarket", ciertos datos críticos del negocio como precios e inventarios requieren una integridad estricta. Para garantizar esto, el equipo implementó reglas de validación JSON Schema directamente en el motor de la base de datos a nivel servidor, actuando como una última línea de defensa independiente de la lógica de la aplicación.

Utilizando el operador `$jsonSchema` al momento de crear las colecciones, se definieron tipos de datos obligatorios y restricciones lógicas. En la colección productos, se estableció que el campo `price` debe ser siempre un número positivo, evitando errores comunes donde un precio

podría guardarse como texto o un valor negativo, lo cual rompería los cálculos del dashboard de ventas. Los campos obligatorios incluyen name, sku y price, con tipos de datos específicos como double para precios y restricciones de minimum: 0. Para la colección users, se aplicó una validación de patrón mediante expresiones regulares en el campo email para asegurar que cumple con el formato estándar antes de ser insertado.

La configuración de validationAction en modo error (estricto) para nuevas inserciones significa que si la aplicación intenta guardar un producto sin precio o con un formato incorrecto, MongoDB rechazará la operación de escritura inmediatamente, lanzando una excepción que el desarrollador debe manejar. Esta decisión asegura que la base de datos no se contamine con datos inconsistentes durante la migración o el desarrollo de nuevas características, cumpliendo con el estándar de integridad exigido en el proyecto. Con esta combinación de embedding para lecturas rápidas, referencing para entidades independientes y schema validation para la integridad, se construyó una arquitectura robusta optimizada para las consultas de agregación complejas.

Proceso de Transformación y Validación

El proceso de transformación de datos se implementó mediante un script Python especializado que procesa el dataset CSV original y genera archivos JSON válidos para cada colección. El script aborda múltiples desafíos de calidad de datos identificados durante el análisis exploratorio, incluyendo limpieza de valores monetarios, normalización de cadenas de texto y sanitización de identificadores. Para los productos, el script agrupa registros por product_id para

evitar duplicados, procesa categorías jerárquicas separadas por pipes, y valida que los precios sean numéricos positivos dentro del rango permitido por el esquema.

La transformación de usuarios requirió especial atención debido a inconsistencias en los nombres dentro del dataset original. Se implementó una función de sanitización que valida la longitud mínima de nombres, verifica la presencia de caracteres alfabéticos para evitar valores como asteriscos o números, y asigna un valor por defecto de "Amazon Customer" cuando los datos no cumplen los criterios de calidad. Esta decisión asegura que todos los registros de usuarios cumplan con la validación de esquema que requiere nombres de al menos dos caracteres válidos.

Para las reseñas, el script implementa detección de duplicados mediante un conjunto de seguimiento de IDs ya procesados, evitando que múltiples reseñas con el mismo identificador se inserten en la colección. La generación de ventas simuladas respeta estrictamente el patrón de formato SALE-YYYY-NNNNNN definido en el esquema de validación, asegura que solo se referencien productos con precios válidos, y asigna aleatoriamente métodos de pago y estados de pedido de las listas enumeradas permitidas.

Posterior a la transformación, se ejecutó un script de validación Python que verifica la integridad referencial entre colecciones antes de la importación a MongoDB Atlas. Este script carga los cuatro archivos JSON generados, valida que los precios en productos sean no negativos, verifica el formato de emails mediante expresiones regulares, comprueba que los rangos de rating estén entre 0 y 5, y detecta referencias huérfanas donde reseñas o ventas apuntan a productos o usuarios inexistentes. El proceso de validación reporta estadísticas detalladas

incluyendo conteo de registros válidos, número de duplicados detectados, y lista de inconsistencias encontradas, proporcionando un reporte de calidad de datos antes de la carga a la plataforma de producción.

Ingesta de Datos en MongoDB Atlas

La carga de datos al clúster M0 en MongoDB Atlas se realizó exclusivamente mediante MongoDB Compass, la interfaz gráfica oficial de MongoDB. Previo a la importación, se configuró el clúster en la región más cercana geográficamente para optimizar la latencia de conexión. La conexión al clúster se estableció utilizando la cadena de conexión proporcionada por Atlas, verificando la autenticación y los permisos de escritura sobre la base de datos globalmarket.

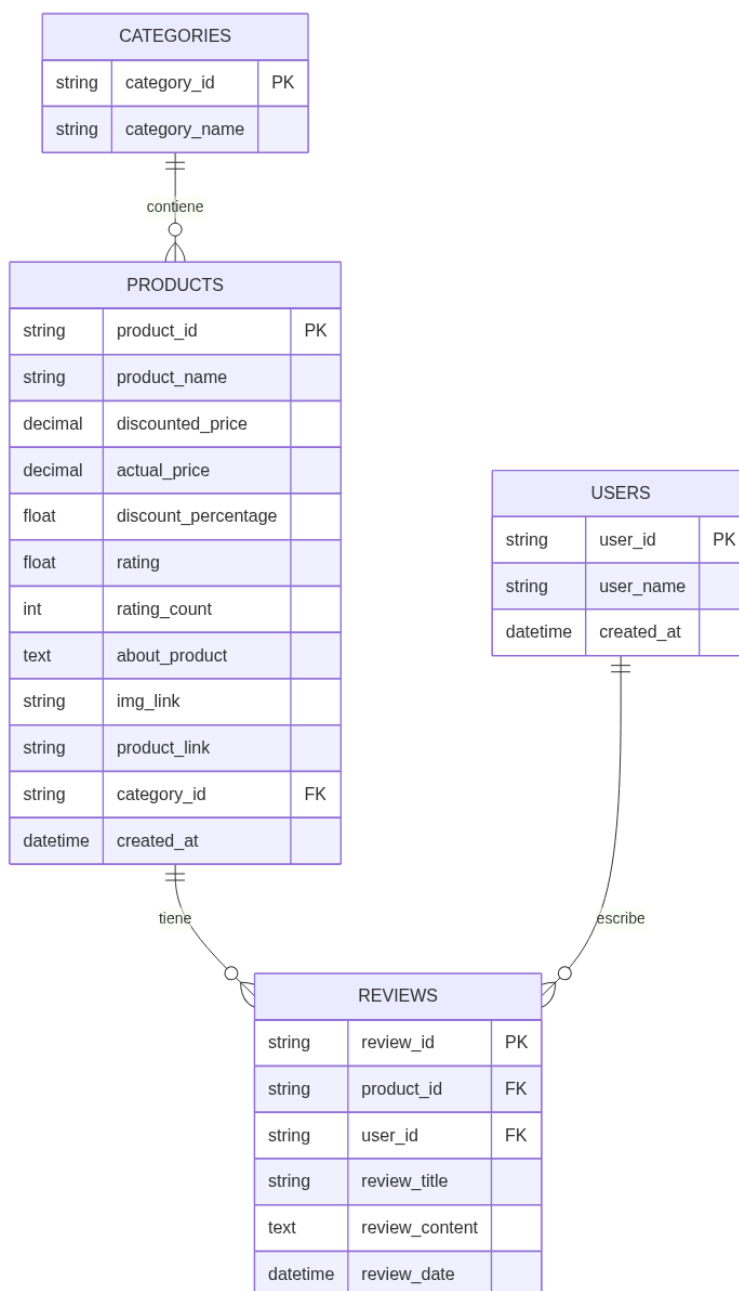
El proceso de importación se ejecutó colección por colección, iniciando con las entidades maestras (productos y usuarios) antes de cargar las transaccionales (reseñas y ventas) para minimizar errores de integridad referencial. En MongoDB Compass, se utilizó la funcionalidad "Add Data" con la opción "Import JSON or CSV file" para cada uno de los cuatro archivos JSON generados. Durante la importación, Compass validó automáticamente la estructura JSON y reportó el número de documentos insertados exitosamente.

Inmediatamente después de cada importación, se verificó la correcta aplicación de las reglas de validación de esquema intentando insertar manualmente documentos inválidos desde la interfaz de Compass. Por ejemplo, se intentó crear un producto con precio negativo y un usuario

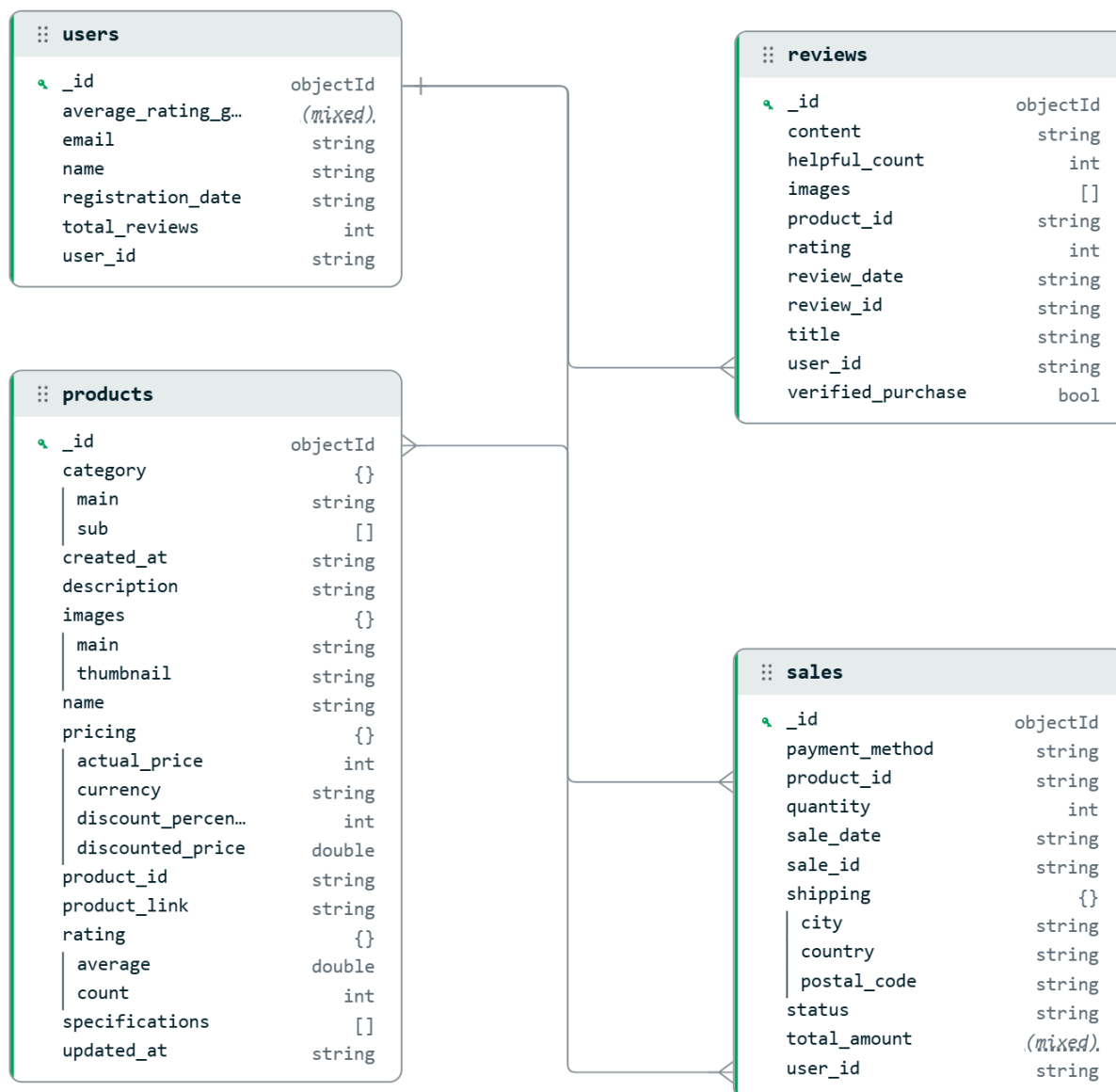
con email mal formado, confirmando que MongoDB rechazaba estas operaciones con mensajes de error descriptivos. Esta verificación en tiempo real demostró que las validaciones de esquema están activas y operativas en el clúster de producción.

Finalmente, se ejecutaron consultas de conteo y muestreo aleatorio en cada colección para verificar que el volumen de datos coincide con los archivos de origen y que la estructura de los documentos preserva la jerarquía diseñada. Se confirmó que los arrays embebidos de especificaciones en productos y los sub-documentos de direcciones de envío en ventas mantienen su estructura anidada correctamente. El proceso completo de ingesta tomó aproximadamente 15 minutos para los mil documentos distribuidos entre las cuatro colecciones, completando exitosamente la fase de diseño e ingesta del proyecto.

Modelo de Datos Relacional (Diagrama Entidad-Relación - ERD):



Modelo de Datos NoSQL (MongoDB/JSON Schema):



Nota: La colección “sales” actúa como el núcleo transaccional del ecosistema GlobalMarket Analytics. Aunque ha sido generada sintéticamente, su estructura está diseñada para representar fielmente el flujo de compra, vinculando las entidades de users y products para permitir un análisis profundo del comportamiento comercial.

Fase II - Desarrollo

Inteligencia de Negocio con Aggregation Framework

Tras asegurar la integridad y la ingesta exitosa de los datos en MongoDB Atlas, pasamos a la capa de análisis. El objetivo fue transformar los datos brutos en insights accionables para la toma de decisiones estratégicas en "GlobalMarket".

A diferencia de las consultas simples (find), el equipo implementó Pipelines de Agregación. Esta herramienta permite procesar documentos a través de una secuencia de etapas (stages), donde la salida de una etapa se convierte en la entrada de la siguiente, permitiendo transformaciones complejas, uniones de colecciones y cálculos estadísticos en tiempo real.

A continuación, se detallan las tres operaciones analíticas principales desarrolladas para el dashboard ejecutivo.

Reporte de Rendimiento: Ventas Mensuales por Categoría

¿Qué categorías de productos generan mayores ingresos y cuál es su tendencia mensual?, Dado que la colección sales contiene el monto de la transacción y la fecha, pero la colección products contiene la categoría, fue necesario realizar una operación de unión (left outer join).

El pipeline se diseñó en cuatro pasos lógicos:

- ❖ **Conversión de Fechas:** Transformar el campo `sale_date` de string a objeto fecha para extraer el mes.
- ❖ **Enriquecimiento (\$lookup):** Cruzar las ventas con el catálogo de productos para obtener la categoría `main`.
- ❖ **Agrupación (\$group):** Sumar el `total_amount` segmentado por mes y categoría.
- ❖ **Proyección (\$project):** Formatear la salida para el frontend del dashboard.

Análisis de Reputación: Productos "Estrella" Verificados

¿Cuáles son los productos que no solo tienen buena calificación, sino que tienen suficiente volumen de reseñas para que la calificación sea estadísticamente confiable? El equipo identificó que ordenar simplemente por `rating.average` traía productos con una sola reseña de 5 estrellas, sesgando el análisis. Se aplicó un filtro estricto de credibilidad usando `$match` sobre el campo `rating.count`. Este pipeline permite identificar los verdaderos líderes del mercado para destacarlos.

Segmentación de Mercado: Bucket Pattern por Precios

¿Cómo se distribuye nuestro inventario en términos de gamas de precios (Económico, Estándar, Premium) para ajustar la estrategia de marketing?, En lugar de procesar esta lógica en el código de la aplicación (Python/Node.js), se utilizó el operador `$bucket` para clasificar los productos directamente en la base de datos. Esto es mucho más eficiente que traer todos los productos y recorrerlos con un bucle `for`. Se definieron los límites basándose en la moneda base (USD/INR) del campo `pricing.discounted_price`.

Fase III - Optimización

Tras completar la implementación de los pipelines de agregación pasamos al rendimiento a escala. Si bien las consultas analíticas arrojaban los datos correctos, el análisis de ejecución reveló que la base de datos estaba realizando "escaneos de colección completos" (COLLSCAN), una operación costosa que implica leer cada documento en disco. Para una plataforma de comercio electrónico proyectada a millones de transacciones, esta latencia es inaceptable.

En esta fase, el equipo ejecutó una estrategia de optimización dual: la implementación de Atlas Search para mejorar la experiencia de usuario en el descubrimiento de productos, y la creación de Índices estratégicos para reducir los tiempos de respuesta.

Atlas Search: La Experiencia de Búsqueda "Fuzzy"

El motor de búsqueda nativo de MongoDB (basado en expresiones regulares \$regex) resultó insuficiente para los patrones de búsqueda modernos. Los usuarios de comercio electrónico a menudo cometen errores tipográficos (ej. buscar "lpatop" en lugar de "laptop") o buscan por descripciones semánticas.

Implementación de Lucene

Para solucionar esto, se habilitó Atlas Search, una integración nativa de Apache Lucene dentro del clúster de MongoDB Atlas. A diferencia de los índices tradicionales de base de datos

que buscan coincidencias exactas, Atlas Search utiliza índices invertidos que permiten análisis lingüístico. Se definió una configuración de índice JSON personalizada con las siguientes características:

- ❖ **Mapeo Estático:** Se deshabilitó el mapeo dinámico para optimizar el espacio, indexando únicamente los campos críticos: name, description y category.
- ❖ **Analizadores de Texto:** Se aplicó lucene.standard al campo name para tokenización general. Se aplicó lucene.english al campo description, lo que permite que una búsqueda de "running" encuentre productos descritos como "ran" o "runs" (stemming).
- ❖ **Fuzzy Search:** Esta configuración habilita la tolerancia a fallos tipográficos. Una búsqueda de "Smsung" ahora devuelve correctamente los teléfonos "Samsung", una capacidad vital para maximizar la conversión de ventas que el motor SQL anterior no poseía.

Estrategia de Indexación: El Patrón ESR

Para optimizar las agregaciones complejas desarrolladas en la Fase II, el equipo adoptó la metodología ESR (Equality, Sort, Range), que dicta el orden óptimo de los campos en un índice compuesto.

Optimización de Consultas Críticas

El análisis de las operaciones más frecuentes dictó la creación de los siguientes índices:

- ❖ Integridad de Joins (\$lookup): Problema: El reporte de ventas cruzaba miles de transacciones con productos, causando lentitud.
- ❖ Solución: Se creó un índice en product_id tanto en la colección products (como único) como en sales. Esto transformó la etapa \$lookup de una operación de fuerza bruta a una búsqueda indexada instantánea.

"Top Productos" (Índice Compuesto)

Buscar productos con más de 50 reseñas (Range) y ordenarlos por calificación (Sort). Se implementó el índice idx_products_rating_analysis: { "rating.count": 1, "rating.average": -1 }. Este índice permite a MongoDB filtrar y ordenar los documentos sin necesidad de cargarlos en memoria para ordenarlos (blocking sort), reduciendo drásticamente el uso de CPU.

Filtros de Navegación (Categoría + Precio)

Usuarios navegando una categoría específica y filtrando por rango de precios. Índice compuesto idx_category_price que cubre ambos predicados, permitiendo que la página de categoría cargue instantáneamente incluso bajo carga pesada.

Análisis de Rendimiento: Explain Plan

Para validar empíricamente las mejoras, se utilizó la herramienta .explain("executionStats") antes y después de la indexación. Los resultados fueron concluyentes:

Caso de Estudio: Filtrado de Ventas Completadas

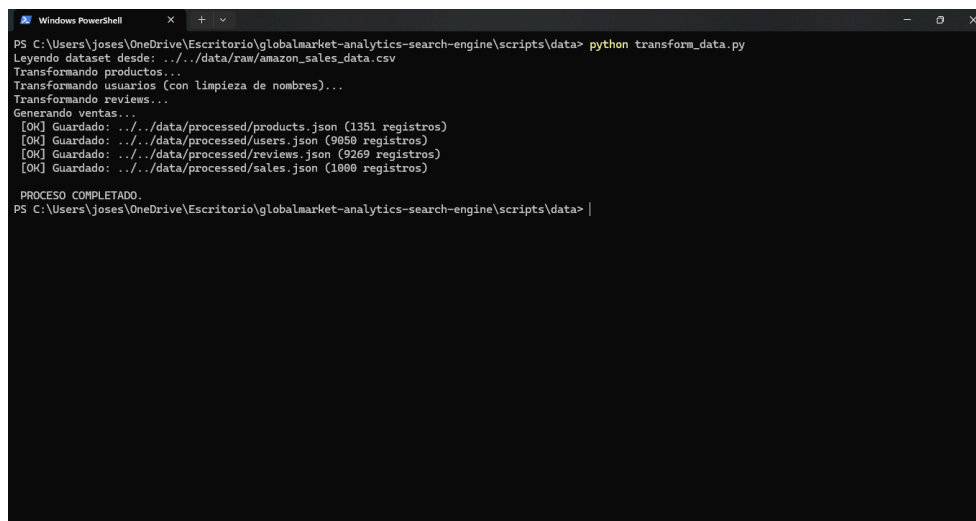
❖ Antes de la Optimización:

- Stage: COLLSCAN
- Documentos Examinados: > 1,000 (Totalidad de la colección)
- Diagnóstico: El motor leía todas las ventas, incluso las canceladas o pendientes, desperdiciando I/O.

❖ Después de la Optimización (idx_sales_status_date):

- Stage: IXSCAN (Escaneo de Índice)
- Documentos Examinados: 490 (Solo los relevantes)
- Conclusión: La consulta ahora es "selectiva". El motor salta directamente a los punteros de las ventas "completed" en el índice y las recupera ordenadas por fecha, eliminando la necesidad de ordenar en memoria.

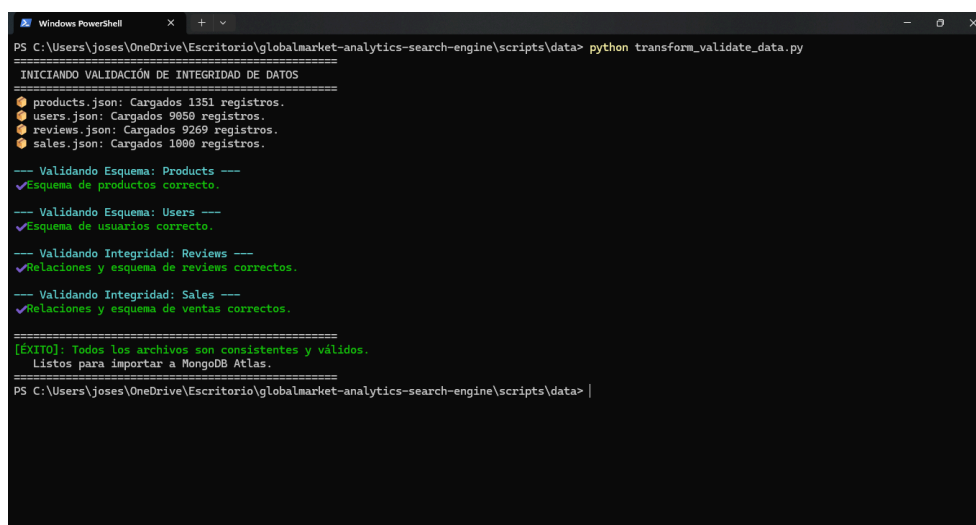
Screenshots



```
Windows PowerShell
PS C:\Users\joses\OneDrive\Escritorio\globalmarket-analytics-search-engine\scripts\data> python transform_data.py
Leyendo dataset desde: ../../data/raw/amazon_sales_data.csv
Transformando productos...
Transformando usuarios (con limpieza de nombres)...
Transformando reviews...
Generando ventas...
[OK] Guardado: ../../data/processed/products.json (1351 registros)
[OK] Guardado: ../../data/processed/users.json (9050 registros)
[OK] Guardado: ../../data/processed/reviews.json (9269 registros)
[OK] Guardado: ../../data/processed/sales.json (1000 registros)

PROCESO COMPLETADO.
PS C:\Users\joses\OneDrive\Escritorio\globalmarket-analytics-search-engine\scripts\data> |
```

Ejecución exitosa del script ETL (transform_data.py) en Python. El proceso realiza la ingestión del dataset crudo amazon_sales_data.csv, aplica limpieza de nombres de usuario y normalización, resultando en la generación de cuatro archivos JSON estructurados (Productos, Usuarios, Reviews y Ventas) listos para la migración a la base de datos.



```
Windows PowerShell
PS C:\Users\joses\OneDrive\Escritorio\globalmarket-analytics-search-engine\scripts\data> python transform_validate_data.py

=====
INICIANDO VALIDACIÓN DE INTEGRIDAD DE DATOS
=====
● products.json: Cargados 1351 registros.
● users.json: Cargados 9050 registros.
● reviews.json: Cargados 9269 registros.
● sales.json: Cargados 1000 registros.

--- Validando Esquema: Products ---
✓Esquema de productos correcto.

--- Validando Esquema: Users ---
✓Esquema de usuarios correcto.

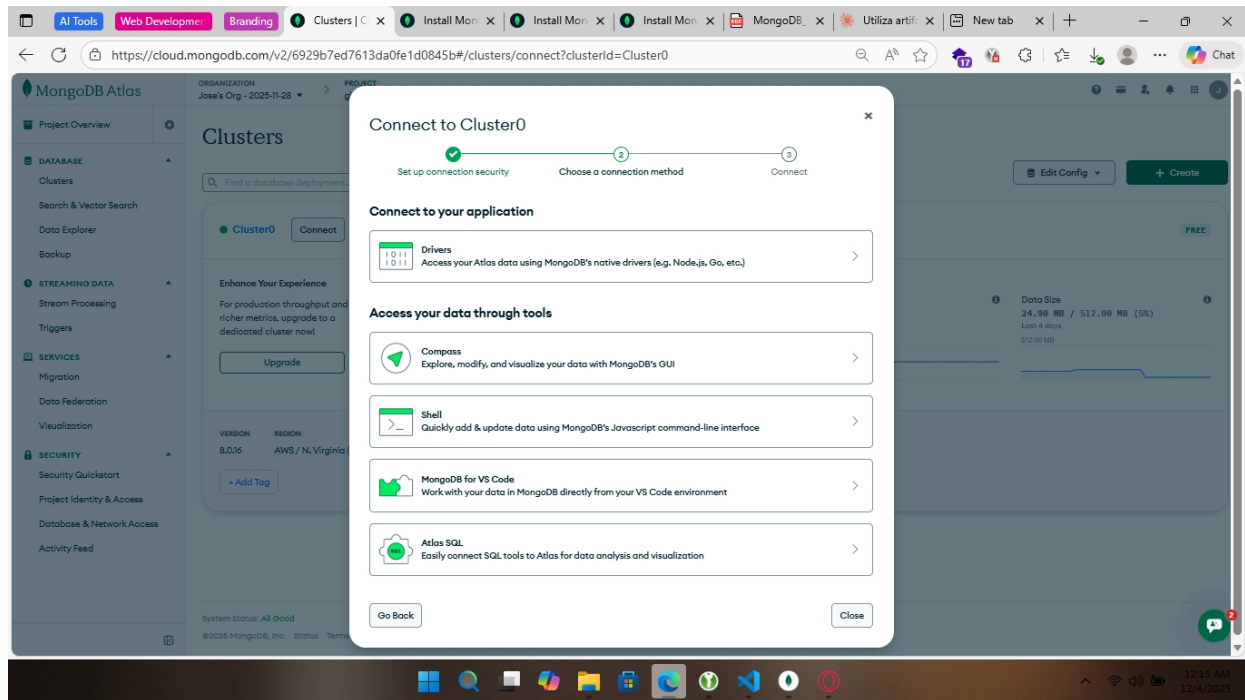
--- Validando Integridad: Reviews ---
✓Relaciones y esquema de reviews correctos.

--- Validando Integridad: Sales ---
✓Relaciones y esquema de ventas correctos.

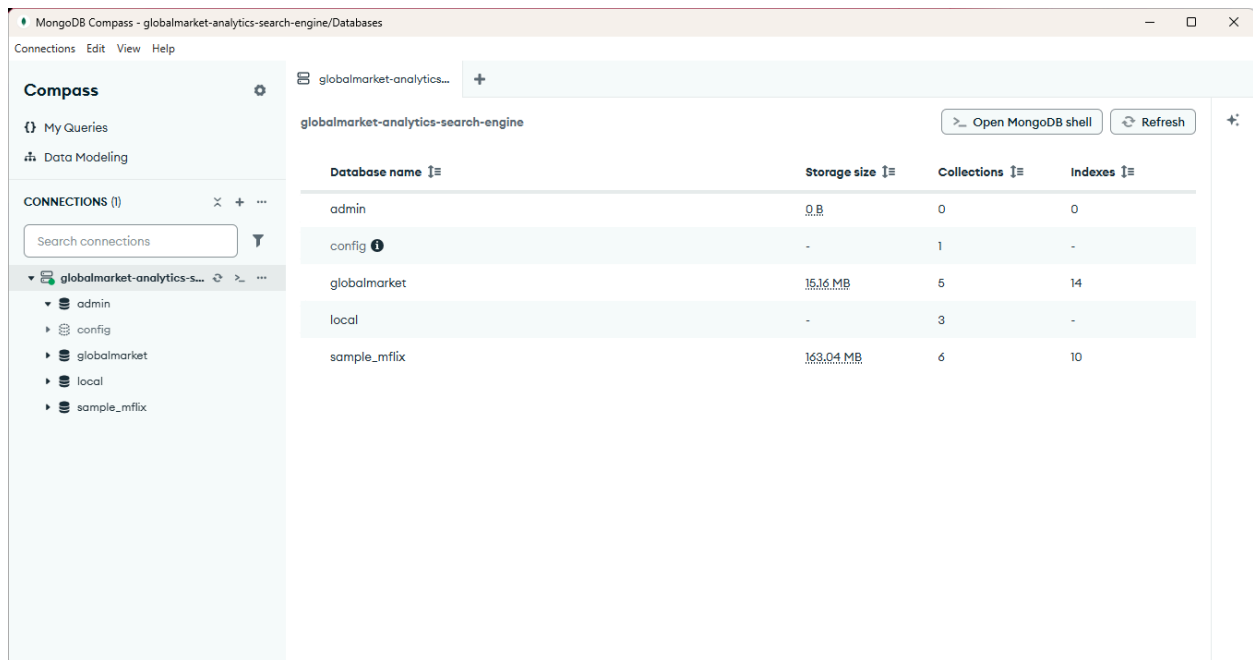
=====
[ÉXITO]: Todos los archivos son consistentes y válidos.
Listos para importar a MongoDB Atlas.
=====
PS C:\Users\joses\OneDrive\Escritorio\globalmarket-analytics-search-engine\scripts\data> |
```

Ejecución exitosa del script (transform_validation_data.py) en Python. Para la validacion de la correcta limpieza de datos.

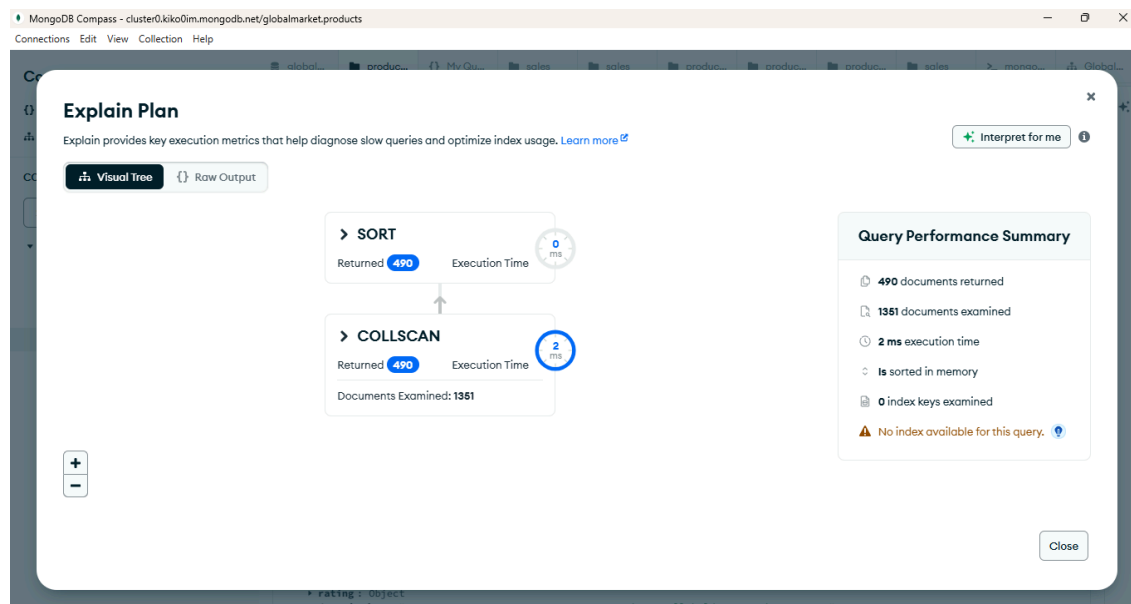
Conexión:



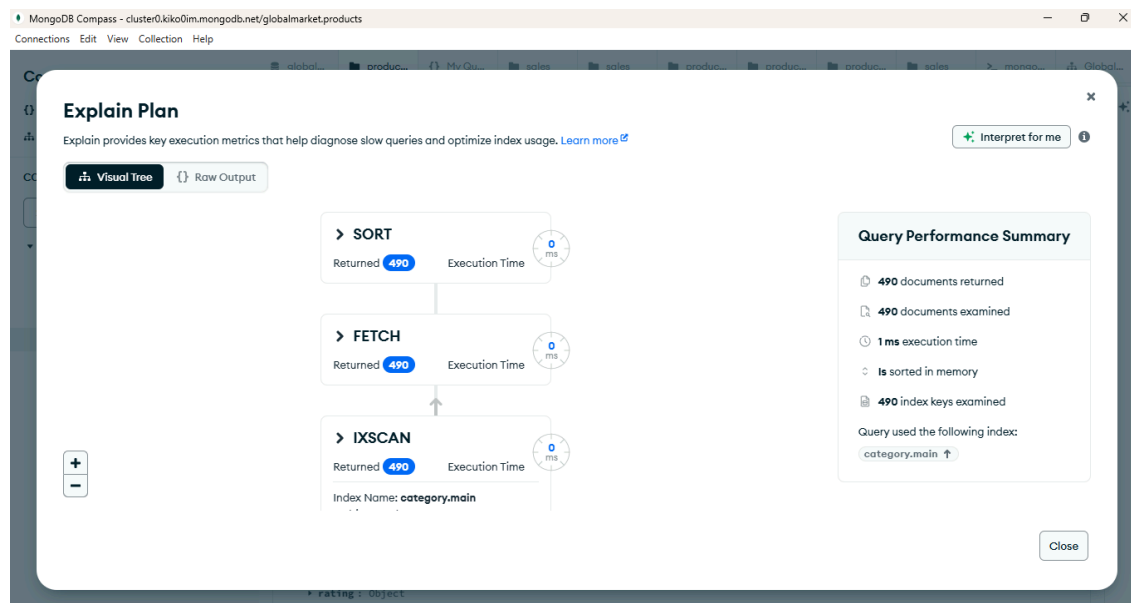
Conexion de Mongo Atlas con Mongo Compass



Performance:



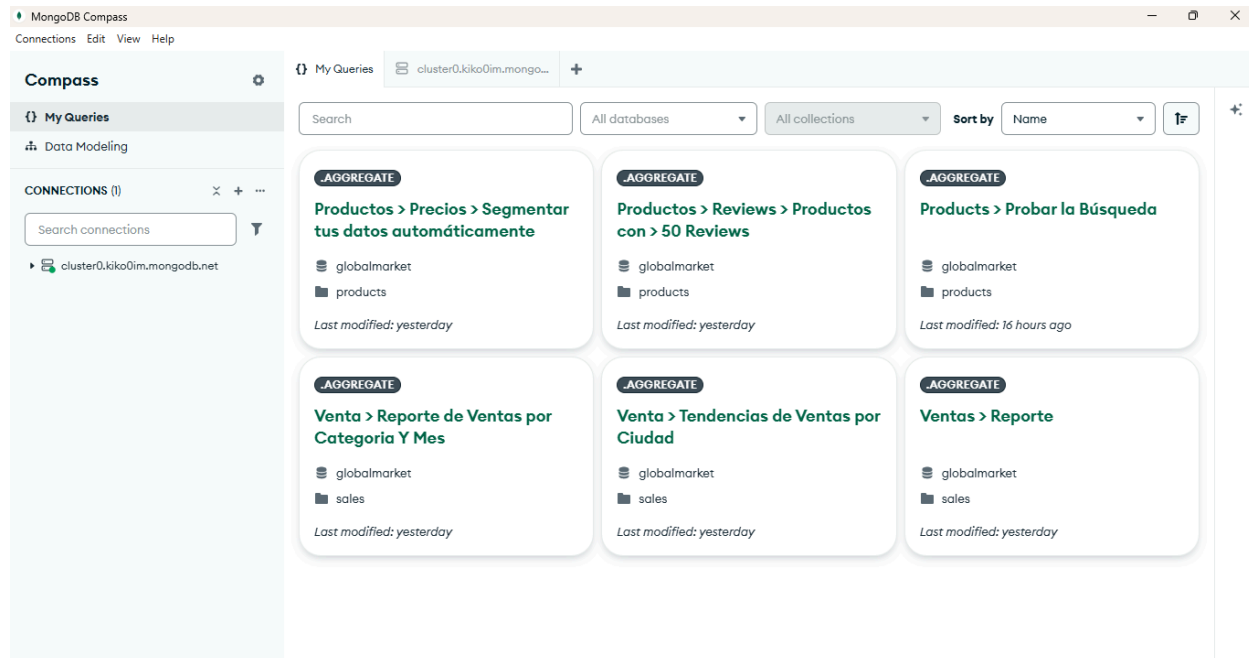
Query Performance COLLSCAN (No optimizada)



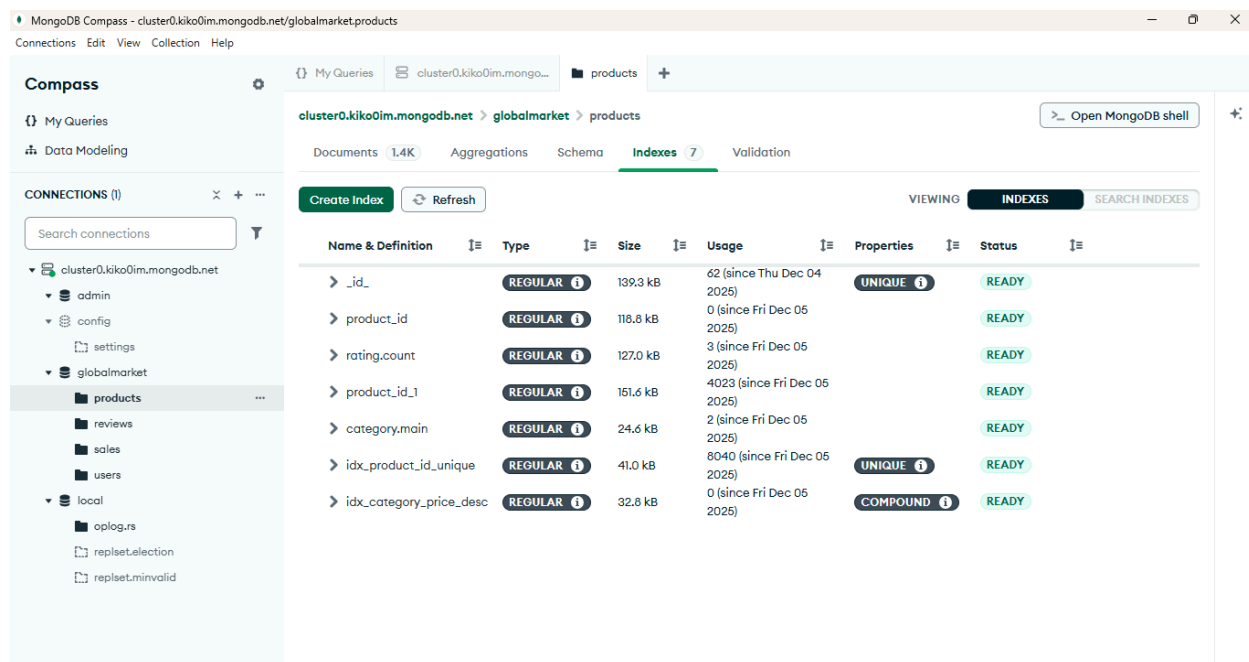
Query Performance IXSCAN

(Misma consulta nivel de documentos revisados baja de 1361 a 490 para las categorías)

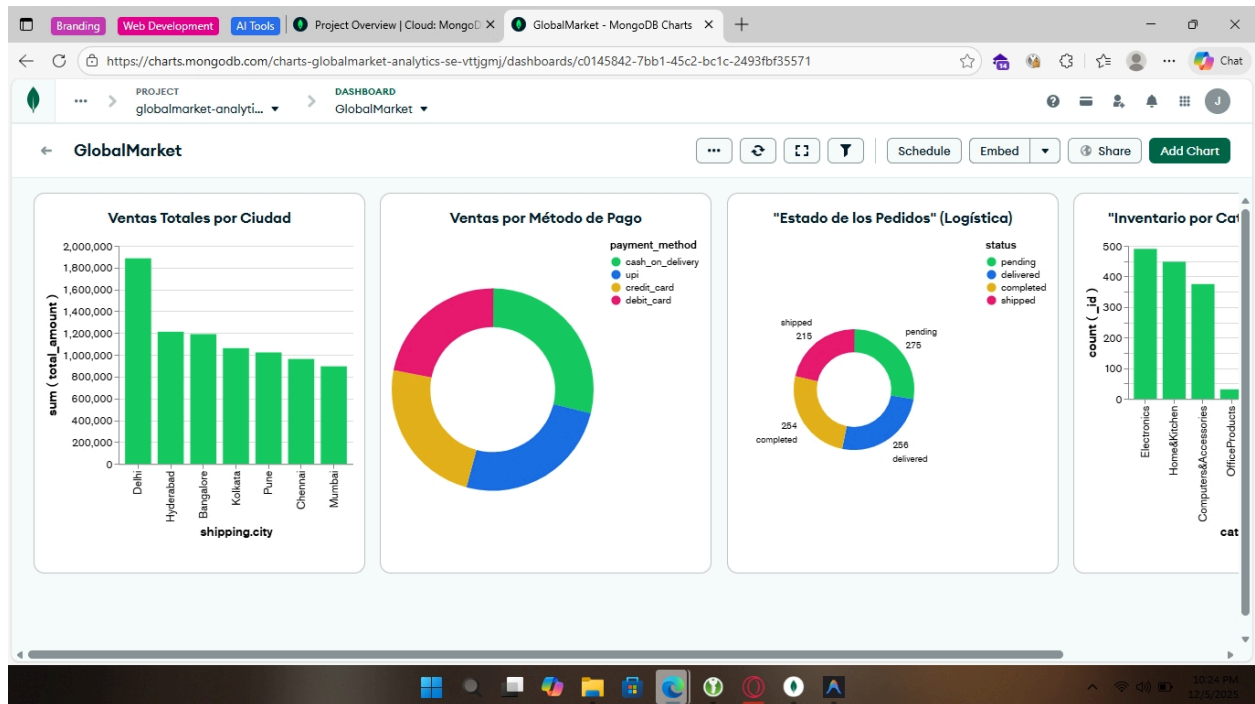
Queries:



Index:



Mongo Chart:



Conclusiones

El desarrollo del proyecto "GlobalMarket Analytics & Search Engine" ha permitido demostrar la eficacia de migrar una arquitectura relacional rígida a un modelo documental flexible utilizando MongoDB Atlas. A través de la implementación, se validó que el diseño de esquema en NoSQL no se basa en la normalización pura, sino en los patrones de acceso a los datos. La decisión estratégica entre embeber (embedding) y referenciar (referencing) resultó crucial para optimizar la lectura de catálogos de productos y el historial de ventas. El uso del Aggregation Framework sustituyó eficazmente las consultas SQL complejas, permitiendo generar reportes analíticos de negocio (como ventas por categoría y segmentación de clientes) con alto rendimiento. Además, la integración de Atlas Search proporcionó una experiencia de búsqueda moderna y "difusa" que hubiera sido costosa de implementar en un entorno SQL tradicional.

Finalmente, el análisis de rendimiento mediante "Explain Plan" evidenció cómo una estrategia de indexación correcta reduce drásticamente los tiempos de respuesta, confirmando que MongoDB es la solución escalable que la startup "GlobalMarket" necesitaba para soportar su crecimiento y la variabilidad de sus datos.

Referencias

Jose-Miserol. (s. f.). *GitHub - jose-miserol/globalmarket-analytics-search-engine*. GitHub.

<https://github.com/jose-miserol/globalmarket-analytics-search-engine>

MongoDB Charts. (s. f.).

<https://charts.mongodb.com/charts-globalmarket-analytics-se-vttjgmj/public/dashboards/c0145842-7bb1-45c2-bc1c-2493fbf35571>

NoSQL vs SQL Databases. (s. f.). MongoDB.

<https://www.mongodb.com/resources/basics/databases/nosql-explained/nosql-vs-sql>

Relational vs. Non-Relational databases. (s. f.). MongoDB.

<https://www.mongodb.com/resources/compare/relational-vs-non-relational-databases>

Team, M. D. (s. f.-a). *Embedded Data in Your MongoDB Schema*. Database Manual - MongoDB

Docs. <https://www.mongodb.com/docs/manual/data-modeling/embedding/>

Team, M. D. (s. f.-b). *Reference Data in Your MongoDB Schema*. Database Manual - MongoDB

Docs. <https://www.mongodb.com/docs/manual/data-modeling/referencing/>

Team, M. D. (s. f.-c). *Schema validation*. Database Manual - MongoDB Docs.

<https://www.mongodb.com/docs/manual/core/schema-validation/>

Team, M. D. (s. f.-a). *Aggregation Operations*. Database Manual - MongoDB Docs.

<https://www.mongodb.com/docs/manual/aggregation/>